



UNS
UNIVERSIDAD
NACIONAL DEL SANTA

CURSO DE APLICACIONES MÓVILES

Desarrolla. Conecta. Innova.
Lleva tus ideas al siguiente nivel.



Flutter



Kotlin



React Native



Android



iOS

```
import 'package:flutter  
class HomePage extends  
@override  
Widget build(BuildContext  
return Scaffold(  
  appBar: AppBar(  
    body: Center(  
      child: Text(  
        'Hola, Mundo!!  
    )  
  )  
)  
)
```



**DESARROLLO
MULTIPLATAFORMA**
Crea apps para Android
e iOS con un solo código.



**INTERFACES
MODERNAS**
Diseña experiencias
atractivas y funcionales.



**CONECTIVIDAD
Y SERVICIOS**
Integra APIs, bases de datos
y servicios en la nube.



**SEGURIDAD
Y RENDIMIENTO**
Desarrolla aplicaciones
seguras, rápidas y
optimizadas.



**PUBLICACIÓN
Y DESPLIEGUE**
Publica tu app en Google
Play y App Store.



DESARROLLO
DE APPS



UI/UX
MÓVIL



APIS Y
SERVICIOS



BASES DE
DATOS



PRUEBAS Y
DEPLOY



Docente:

Dr. Mirko Manrique Ronceros

INTRODUCCIÓN

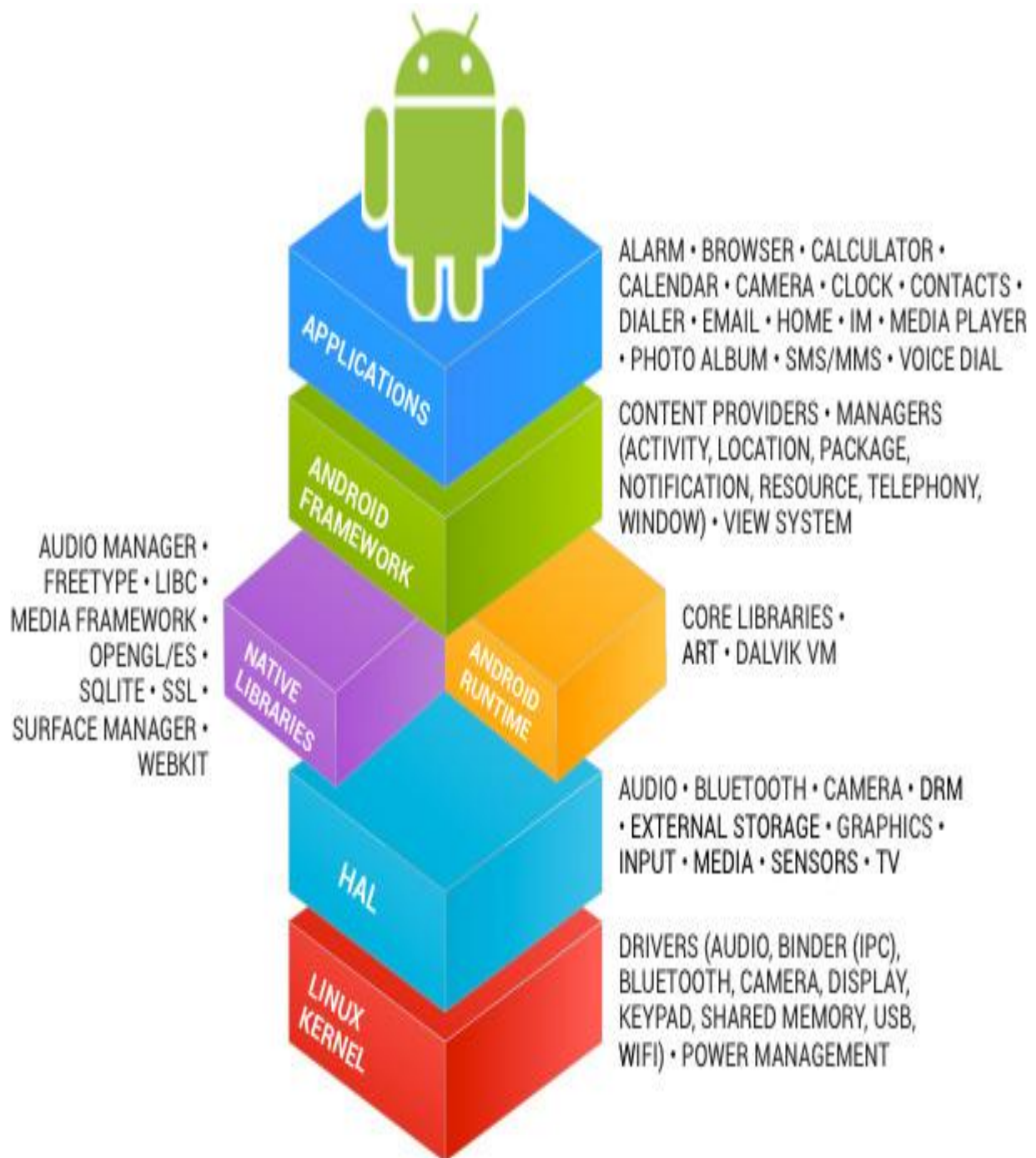
En la actualidad, el sistema operativo Android se ha consolidado como una de las plataformas más utilizadas en el desarrollo de aplicaciones móviles a nivel mundial. Su popularidad se debe no solo a su carácter abierto y flexible, sino también a la robusta arquitectura sobre la cual está construido. Comprender la arquitectura de la plataforma Android es fundamental para cualquier desarrollador, ya que esta define cómo interactúan sus diversos componentes: desde el núcleo del sistema operativo hasta las aplicaciones que el usuario final ejecuta. Cada capa —el núcleo de Linux, las bibliotecas nativas, el entorno de ejecución ART, el marco de aplicaciones y las aplicaciones mismas— cumple un rol esencial en el funcionamiento del sistema.

En este contexto, las librerías de Android desempeñan un papel clave al proporcionar funcionalidades reutilizables que optimizan el proceso de desarrollo. Existen tanto librerías oficiales como de terceros, orientadas a aspectos diversos como la interfaz gráfica, el acceso a redes, la persistencia de datos, la gestión de imágenes, la animación o la seguridad. El uso eficaz de estas librerías permite crear aplicaciones más robustas, eficientes y con mejores experiencias de usuario, al mismo tiempo que se reduce significativamente el tiempo de desarrollo.

Complementando estos elementos, el entorno de aplicación de Android brinda a los desarrolladores un ecosistema integral compuesto por herramientas, frameworks, emuladores y servicios que permiten diseñar, construir, probar y desplegar aplicaciones móviles de manera profesional. Desde Android Studio hasta herramientas de depuración y monitoreo, el entorno de desarrollo de Android está diseñado para apoyar todas las etapas del ciclo de vida de una aplicación, con una fuerte integración de lenguajes como Java y Kotlin.

I. ARQUITECTURA DE LA PLATAFORMA ANDROID

La arquitectura de Android es una pila de componentes de software que se organiza en varias capas. Cada capa se comunica con la capa inferior y proporciona servicios a la capa superior. Esta arquitectura modular permite una gran flexibilidad, reutilización de componentes y portabilidad a diferentes dispositivos. Las principales capas de la arquitectura de Android son:



1) Kernel de Linux

a) Concepto: En la base de la arquitectura de Android se encuentra el kernel de Linux. Android no es simplemente una distribución de Linux tradicional, pero utiliza el kernel de Linux como su núcleo. Google ha realizado modificaciones significativas en el kernel de Linux para adaptarlo a las necesidades de un sistema operativo móvil, como la gestión de energía, la administración de memoria, la seguridad y el soporte para hardware específico de dispositivos móviles.

b) Características y Funciones Clave:

- **Gestión de Procesos:** El kernel de Linux es responsable de la creación, planificación y gestión de los procesos que se ejecutan en el dispositivo.
- **Gestión de Memoria:** Administra la asignación y liberación de memoria para las aplicaciones y el sistema operativo. Android utiliza mecanismos como el "low memory killer" para liberar memoria cuando el sistema está bajo presión.
- **Gestión de Redes:** Proporciona la pila de protocolos de red (TCP/IP) y gestiona las conexiones de red (Wi-Fi, Bluetooth, datos móviles).
- **Sistema de Archivos:** Soporta varios sistemas de archivos, como EXT4, y gestiona el acceso al almacenamiento del dispositivo.
- **Controladores de Dispositivos (Drivers):** El kernel contiene los drivers que permiten al sistema operativo interactuar con el hardware específico del dispositivo (pantalla táctil, cámara, sensores, etc.). Los fabricantes de hardware contribuyen con drivers específicos para sus dispositivos.
- **Seguridad:** El kernel implementa mecanismos de seguridad a nivel del sistema, como permisos y aislamiento de procesos.

- **Gestión de Energía:** Optimiza el uso de la batería mediante técnicas como la suspensión de procesos inactivos y la gestión de la frecuencia del CPU.
 - **Binder IPC (Inter-Process Communication):** Un mecanismo crucial para la comunicación eficiente entre los diferentes procesos y servicios que se ejecutan en Android.
- c) **Ejemplos:** La interacción de una aplicación con la cámara del dispositivo pasa a través de las APIs de Android, que a su vez utilizan los drivers de la cámara gestionados por el kernel de Linux.

2) HAL (Hardware Abstraction Layer)

a) **Concepto:** La capa HAL actúa como una interfaz estándar entre el framework de Android (la capa superior) y el hardware específico del dispositivo (que está controlado por el kernel de Linux). El HAL define una interfaz abstracta para los componentes de hardware, lo que permite que el framework de Android interactúe con el hardware sin necesidad de conocer los detalles de implementación de cada fabricante.

b) Características y Funciones Clave:

- **Abstracción de Hardware:** Proporciona una capa de abstracción que oculta las diferencias entre las implementaciones de hardware de diferentes fabricantes.
- **Módulos HAL:** El HAL se implementa a través de módulos (bibliotecas dinámicas .so) que son cargados por el sistema Android cuando es necesario. Cada módulo implementa una interfaz estándar para un tipo específico de hardware (cámara, sensor, Wi-Fi, Bluetooth, audio, etc.).

- **Interfaces Estándar:** Google define interfaces HAL estándar para los componentes de hardware comunes. Los fabricantes implementan estas interfaces para que su hardware sea compatible con Android.
 - **Facilita la Portabilidad:** Permite que Android se ejecute en una amplia variedad de hardware, ya que los fabricantes solo necesitan proporcionar las implementaciones de los módulos HAL para sus dispositivos.
- c) **Ejemplos:** Cuando una aplicación quiere acceder a la cámara, interactúa con las APIs del framework de Android. Estas APIs, a través del HAL de la cámara, se comunican con el driver de la cámara específico del hardware en el kernel de Linux.

3) Native C/C++ Libraries

- a) **Concepto:** Esta capa contiene un conjunto de bibliotecas nativas escritas en C y C++ que proporcionan funcionalidades esenciales para los componentes de nivel superior de Android. Estas bibliotecas son utilizadas por el framework de Android a través del HAL.
- b) **Características y Funciones Clave:**
- **Funcionalidades Fundamentales:** Incluyen bibliotecas para gráficos 2D y 3D (Skia, OpenGL ES), códecs multimedia (libvpx, libstagefright), bases de datos (SQLite), seguridad (SSL/TLS, OpenSSL), soporte para lenguajes (libc), etc.
 - **Rendimiento:** Al ser escritas en lenguajes de bajo nivel como C y C++, estas bibliotecas ofrecen un buen rendimiento para tareas intensivas.
 - **Acceso al Hardware (a través del HAL):** Interactúan con el hardware del dispositivo a través de la capa HAL.

c) Ejemplos:

- **Skia:** Utilizada para el renderizado de gráficos 2D en la interfaz de usuario de Android y en el navegador Chrome.
- **OpenGL ES:** Proporciona APIs para el renderizado de gráficos 3D, utilizado en juegos y aplicaciones con gráficos avanzados.
- **SQLite:** Un motor de base de datos ligero utilizado para el almacenamiento de datos estructurados en las aplicaciones.
- **Media Framework (libstagefright):** Soporta la reproducción y grabación de varios formatos de audio y video.

4) Android Runtime (ART)

a) Concepto: El Android Runtime (ART) es el entorno de ejecución para las aplicaciones escritas en los lenguajes de programación de Android (Java y Kotlin). ART es responsable de la ejecución del bytecode Dalvik (en versiones anteriores de Android) o bytecode propio (a partir de Android 5.0 Lollipop).

b) Características y Funciones Clave:

- **Ejecución de Bytecode:** ART toma el bytecode DEX (Dalvik Executable) de las aplicaciones Android y lo ejecuta en el dispositivo.
- **Compilación Ahead-of-Time (AOT) y Just-in-Time (JIT):** ART utiliza una combinación de compilación AOT (donde el bytecode se compila a código máquina nativo cuando se instala la aplicación) y JIT (donde el bytecode se compila durante la ejecución) para optimizar el rendimiento de las aplicaciones. A partir de Android 7.0 Nougat, ART utiliza un enfoque híbrido con perfiles de compilación guiada.

- **Recolección de Basura (Garbage Collection):** ART gestiona la memoria de las aplicaciones y realiza la recolección de basura para liberar la memoria no utilizada.
 - **Soporte para el Lenguaje Java y Kotlin:** ART proporciona las bibliotecas base de Java y Kotlin necesarias para ejecutar las aplicaciones escritas en estos lenguajes.
 - **Optimización para Dispositivos Móviles:** ART está diseñado para ser eficiente en términos de uso de memoria y batería, características cruciales en dispositivos móviles.
- c) **Ejemplos:** Cuando se ejecuta una aplicación de Android, el código Java o Kotlin se compila a bytecode DEX, y ART es el encargado de ejecutar este bytecode en el dispositivo.

5) Framework de Aplicaciones (Application Framework)

- a) **Concepto:** El framework de aplicaciones proporciona un conjunto de APIs de alto nivel y bloques de construcción que los desarrolladores utilizan para crear aplicaciones de Android. Simplifica el proceso de desarrollo al proporcionar abstracciones y servicios comunes.
- b) **Características y Funciones Clave:**
- **Componentes de la Aplicación:** Define la estructura básica de las aplicaciones de Android a través de componentes como Activities (interfaces de usuario), Services (procesos en segundo plano), Content Providers (gestión de datos compartidos) y Broadcast Receivers (receptores de eventos del sistema).
 - **Gestión de la Interfaz de Usuario:** Proporciona herramientas y clases para crear interfaces de usuario interactivas (Views, Layouts, Widgets).

- **Administración de Recursos:** Permite acceder a recursos de la aplicación como imágenes, archivos de diseño, cadenas de texto, etc.
 - **Administración de Paquetes:** Gestiona la instalación, desinstalación y la información de las aplicaciones.
 - **Administración de la Telefonía:** Proporciona APIs para acceder a las funcionalidades de telefonía (llamadas, SMS).
 - **Administración de la Ubicación:** Permite acceder a la información de ubicación del dispositivo.
 - **Notificaciones:** Proporciona mecanismos para mostrar notificaciones al usuario.
 - **Intents:** Un mecanismo para la comunicación asíncrona entre componentes de la aplicación y otras aplicaciones.
- c) Ejemplos:** Cuando un desarrollador crea una interfaz de usuario con botones y campos de texto, está utilizando las clases y APIs proporcionadas por el framework de aplicaciones (Views, TextView, Button, LinearLayout, etc.). Cuando una aplicación quiere enviar una notificación, utiliza las APIs del NotificationManager del framework.

6) Aplicaciones (Applications)

a) Concepto: Esta es la capa superior de la arquitectura de Android y contiene todas las aplicaciones instaladas en el dispositivo, tanto las aplicaciones del sistema (preinstaladas) como las aplicaciones de terceros instaladas por el usuario.

b) Características:

- **Desarrolladas utilizando el Framework de Aplicaciones:** Todas las aplicaciones se construyen utilizando las APIs y los componentes proporcionados por el framework de aplicaciones.

- **Diversidad de Funcionalidades:** Cubren una amplia gama de funcionalidades, desde herramientas de productividad y comunicación hasta juegos y aplicaciones de entretenimiento.
- **Ejecutadas en el ART:** Se ejecutan dentro del entorno ART.
- **Interactúan con las Capas Inferiores:** Utilizan las APIs del framework de aplicaciones para acceder a los servicios y recursos proporcionados por las capas inferiores (acceso al hardware a través del HAL, almacenamiento a través del kernel, etc.).

c) Ejemplos: Gmail, Chrome, WhatsApp, juegos, aplicaciones de redes sociales, etc.

Ejemplo Práctico

Cuando un usuario abre la aplicación de cámara:

- 1) La aplicación solicita acceso a la cámara a través del framework de aplicaciones.
- 2) El framework utiliza las bibliotecas nativas para comunicarse con el hardware.
- 3) Las solicitudes se transmiten al núcleo de Linux, que interactúa con los controladores de hardware para activar la cámara.
- 4) La imagen capturada se procesa y se muestra en la interfaz de usuario.

Este flujo demuestra cómo las diferentes capas de la arquitectura de Android trabajan conjuntamente para proporcionar funcionalidades a las aplicaciones.

II. LIBRERÍAS DE ANDROID

En el ecosistema de desarrollo de Android, una "librería" se refiere a un conjunto de código (clases, métodos, recursos, etc.) empaquetado de forma reutilizable que los desarrolladores pueden incluir en sus proyectos para extender la funcionalidad de sus aplicaciones sin tener que escribir todo desde cero. Estas librerías pueden ser proporcionadas por Google (dentro del Android SDK), por la comunidad de desarrolladores de código abierto o por empresas de terceros.

Las librerías simplifican tareas comunes como la manipulación de datos, la conexión a redes, la gestión de la interfaz de usuario, el procesamiento de imágenes, la integración con servicios web, y mucho más. Utilizar librerías bien diseñadas no solo acelera el desarrollo, sino que también contribuye a la estabilidad y el rendimiento de las aplicaciones.

1) **Librerías del Android SDK:**

Estas son las librerías fundamentales proporcionadas por Google como parte del Android Software Development Kit (SDK). Ofrecen la base para construir cualquier aplicación de Android.

a) AndroidX: Es una refactorización y mejora de la biblioteca de soporte original de Android. AndroidX separa los componentes del SDK de Android del sistema operativo, lo que permite actualizaciones más frecuentes e independientes. Incluye una amplia gama de librerías para:

- **AppCompat:** Proporciona compatibilidad con las últimas funciones de la interfaz de usuario en versiones antiguas de Android.
- **RecyclerView:** Un componente flexible y eficiente para mostrar listas y cuadrículas de datos.
- **ConstraintLayout:** Un diseño de ViewGroup potente y flexible para crear interfaces de usuario complejas.

- **Material Components:** Implementación de los lineamientos de diseño Material Design de Google.
- **Navigation Component:** Un framework para implementar la navegación dentro de una aplicación.
- **Room Persistence Library:** Una capa de abstracción sobre SQLite para facilitar el almacenamiento de datos persistentes.
- **ViewModel y LiveData:** Componentes para la gestión del estado de la interfaz de usuario y la comunicación entre componentes.
- **WorkManager:** Una API para programar tareas en segundo plano que deben ejecutarse incluso si la aplicación se cierra o el dispositivo se reinicia.

b) Core Android Libraries: Incluyen paquetes fundamentales como `android.app` (gestión del ciclo de vida de las aplicaciones), `android.content` (interacción entre aplicaciones y datos), `android.os` (servicios del sistema operativo), `android.view` (construcción de interfaces de usuario), `android.widget` (componentes de la interfaz de usuario predefinidos), `android.net` (redes), `android.graphics` (gráficos 2D), `android.media` (audio y video), `android.location` (servicios de ubicación), entre otros.

Ejemplo: Para mostrar una lista de elementos, un desarrollador utilizaría la clase `RecyclerView` de la librería `androidx.recyclerview`. Para realizar una solicitud HTTP, podría usar las clases del paquete `java.net` o librerías de terceros construidas sobre ellas.

2) **Librerías de Soporte de Google (fuera de AndroidX):**

Aunque AndroidX es la principal biblioteca de soporte, existen otras librerías proporcionadas por Google para funcionalidades específicas:

- **Firestore SDK:** Un conjunto de librerías para acceder a los servicios de Firestore, como autenticación, bases de datos en tiempo real (Realtime Database y Firestore), almacenamiento en la nube (Cloud Storage), funciones sin servidor (Cloud Functions), aprendizaje automático (ML Kit), análisis (Analytics), mensajería en la nube (Cloud Messaging), etc.

Ejemplo: Para implementar la autenticación de usuarios con Google Sign-In, un desarrollador utilizaría las librerías del Firebase Authentication SDK. Para almacenar datos en la nube de forma NoSQL, usaría las librerías de Firestore.

- **Google Play Services:** Un conjunto de APIs y servicios que permiten a las aplicaciones integrar funcionalidades como mapas (Google Maps Android API), publicidad (AdMob), inicio de sesión con Google (Google Sign-In), juegos (Google Play Games Services), etc.

Ejemplo: Para incrustar un mapa interactivo en una aplicación, se utiliza la Google Maps Android API, parte de Google Play Services. Para mostrar anuncios y monetizar la aplicación, se integra el SDK de AdMob.

3) Librerías de Terceros y de Código Abierto:

Una vasta comunidad de desarrolladores ha creado y compartido innumerables librerías de código abierto que abordan una amplia gama de necesidades. Utilizar estas librerías puede acelerar significativamente el desarrollo y proporcionar funcionalidades robustas y probadas. Algunas categorías comunes incluyen:

a) Networking:

- **Retrofit:** Una librería popular para realizar solicitudes HTTP de tipo seguro.
- **OkHttp:** Una librería HTTP eficiente que sirve como base para Retrofit y otras.

- **Volley:** La librería de networking de Google para Android.

b) Image Loading and Caching:

- **Glide:** Una librería eficiente para cargar y mostrar imágenes, con soporte para caching y animaciones.
- **Picasso:** Otra librería popular para la carga de imágenes, desarrollada por Square.
- **Fresco:** Una potente librería de carga de imágenes de Facebook.

c) Dependency Injection:

- **Dagger/Hilt:** Frameworks para implementar la inyección de dependencias, facilitando la gestión de dependencias y las pruebas unitarias.
- **Koin:** Un framework de inyección de dependencias ligero para Kotlin.

d) JSON Parsing:

- **Gson:** Una librería de Google para convertir objetos Java/Kotlin a JSON y viceversa.
- **Moshi:** Una librería de parsing JSON moderna para Android y Java/Kotlin, desarrollada por Square.

e) Reactive Programming:

- **RxJava/RxAndroid:** Librerías para implementar programación reactiva utilizando Observables.
- **Coroutines (Kotlin):** Aunque no es una librería tradicional, las corrutinas de Kotlin facilitan la escritura de código asíncrono de forma secuencial.

f) Logging:

- **Timber:** Una librería de logging pequeña y útil.

g) UI Enhancements:

- **Lottie:** Para renderizar animaciones basadas en Adobe After Effects en tiempo real.
- **Material Dialogs:** Para crear diálogos personalizados con Material Design.
- **CircularImageView:** Para mostrar imágenes en forma circular.

h) Testing:

- **JUnit:** Framework de pruebas unitarias para Java y Kotlin.
- **Mockito:** Framework para crear objetos mock para pruebas.
- **Espresso:** Framework de pruebas de interfaz de usuario para Android.

Ejemplo: Para realizar una llamada a una API web y obtener datos en formato JSON, un desarrollador podría usar Retrofit junto con Gson para simplificar la solicitud y el parsing de la respuesta. Para mostrar una imagen desde una URL, podría usar Glide para manejar la descarga, el caching y la visualización eficiente.

III. ENTORNO DE APLICACIÓN EN ANDROID

El "entorno de aplicación" en Android se refiere al conjunto de recursos, servicios y la infraestructura proporcionada por el sistema operativo que permite que una aplicación se ejecute correctamente y interactúe con el usuario y el sistema. Este entorno está cuidadosamente gestionado por Android para garantizar la seguridad, la estabilidad y la eficiencia del dispositivo. Los componentes clave del entorno de aplicación incluyen el proceso de la aplicación, la máquina virtual (ART), la gestión de la memoria, la gestión del ciclo de vida, los permisos y la seguridad.

1) Proceso de la Aplicación (Application Process)

a) Concepto: Cada aplicación de Android se ejecuta en su propio proceso Linux aislado. Este aislamiento es un aspecto fundamental de la seguridad en Android, ya que impide que una aplicación interfiera directamente con los recursos de otras aplicaciones o del sistema operativo.

b) Características:

- **Aislamiento:** Cada aplicación tiene su propio espacio de memoria virtual, lo que significa que no puede acceder directamente a la memoria de otras aplicaciones.
- **Identificador Único (UID):** A cada paquete de aplicación se le asigna un identificador de usuario único (UID) por el sistema al instalarse. Todos los procesos de esa aplicación se ejecutan con este UID. Los permisos a nivel de sistema se basan en estos UID.
- **Gestión del Ciclo de Vida por el Sistema:** El sistema operativo Android es responsable de iniciar, detener y gestionar el ciclo de vida de los procesos de las aplicaciones en función de las necesidades del usuario y los recursos del sistema.
- **Comunicación Inter-Procesos (IPC):** Para que las aplicaciones puedan interactuar entre sí (ej., compartir datos, iniciar actividades de otras aplicaciones), Android proporciona mecanismos de Comunicación Inter-Procesos (IPC) bien definidos, como Intents, Content Providers y Binder.

c) empleos: Cuando se abren múltiples aplicaciones (Gmail, un juego, WhatsApp), cada una se ejecuta en su propio proceso separado. Si una aplicación falla, generalmente no afecta a las demás gracias a este aislamiento. Cuando una aplicación quiere compartir una foto con

otra, utiliza un Intent para solicitar la acción, y el sistema Android media la comunicación.

2) Android Runtime (ART - Android Runtime)

a) Concepto: Como se mencionó en la arquitectura de Android, ART es la máquina virtual que ejecuta el bytecode de las aplicaciones Android. Proporciona el entorno en el que el código de la aplicación (escrito en Java o Kotlin) se traduce y se ejecuta en el hardware del dispositivo.

b) Características (Recapitulando y Ampliando):

- **Ejecución de Bytecode DEX:** ART ejecuta el bytecode Dalvik Executable (.dex) que se encuentra dentro del archivo APK de la aplicación.
- **Compilación AOT y JIT (Híbrida):** ART utiliza una combinación de compilación Ahead-of-Time (AOT) (compilación al instalar) y Just-in-Time (JIT) (compilación durante la ejecución) para optimizar el rendimiento. El enfoque híbrido permite un equilibrio entre el inicio rápido de la aplicación y el rendimiento sostenido.
- **Gestión de Memoria (Garbage Collection):** ART gestiona la asignación y liberación de memoria para las aplicaciones. Utiliza algoritmos de recolección de basura para reclamar la memoria que ya no está en uso, lo que ayuda a prevenir fugas de memoria y a mantener la aplicación responsiva.
- **Seguridad:** ART proporciona ciertas características de seguridad al verificar el bytecode y al ejecutar las aplicaciones en un entorno controlado dentro del proceso de la aplicación.
- **Soporte de Bibliotecas Core:** ART proporciona acceso a las bibliotecas core de Java y Kotlin necesarias para que las aplicaciones funcionen.

- c) **Ejemplos:** Cuando se instala una aplicación desde Google Play Store, el bytecode DEX se compila (parcial o totalmente) a código máquina por ART. Durante la ejecución de la aplicación, ART gestiona la memoria de los objetos creados y destruidos.

3) Gestión de la Memoria (Memory Management)

- a) **Concepto:** Android tiene un sistema de gestión de memoria sofisticado diseñado para funcionar eficientemente en dispositivos con recursos limitados (RAM, batería). El sistema operativo debe equilibrar la necesidad de mantener las aplicaciones en ejecución para una experiencia de usuario fluida con la necesidad de liberar memoria para nuevas aplicaciones o procesos del sistema.

b) **Características:**

- **Asignación y Liberación de Memoria:** ART es responsable de asignar memoria a los objetos creados por la aplicación y de liberar esa memoria cuando los objetos ya no son necesarios a través de la recolección de basura.
- **Compartición de Memoria:** Android intenta compartir memoria entre procesos siempre que sea posible (ej., para bibliotecas del sistema o recursos comunes) para reducir el uso total de RAM.
- **Mecanismos para Liberar Memoria:** Cuando la memoria es baja, Android puede recurrir a varias estrategias para liberar memoria:
 - **Cerrar aplicaciones en segundo plano:** El sistema prioriza mantener en ejecución las aplicaciones que el usuario está utilizando activamente. Las aplicaciones en segundo plano que no están realizando tareas importantes pueden ser terminadas para liberar memoria.

- **"Low Memory Killer" (LMK):** Un proceso del sistema que identifica y termina procesos (aplicaciones) menos importantes cuando la memoria es críticamente baja. La importancia se determina en función de la prioridad del proceso y su estado.
 - **Swapping (en algunos dispositivos):** Algunos dispositivos pueden utilizar una partición de swap en el almacenamiento para mover datos de la RAM y liberar memoria, aunque esto es menos común en dispositivos móviles debido al impacto en el rendimiento y la vida útil del almacenamiento.
 - **Restricciones de Memoria por Aplicación:** El sistema impone límites en la cantidad de memoria que una aplicación puede utilizar. Intentar exceder estos límites puede provocar errores OutOfMemoryError.
- c) Ejemplos:** Si una aplicación se minimiza y no realiza ninguna tarea activa, Android puede optar por terminar su proceso si necesita memoria para una aplicación en primer plano. Los desarrolladores deben tener cuidado de liberar referencias a objetos cuando ya no son necesarios para evitar fugas de memoria que puedan llevar a la terminación de su aplicación por el sistema.

4) Gestión del Ciclo de Vida de las Aplicaciones (Application Lifecycle Management)

- a) Concepto:** Android gestiona el ciclo de vida de los componentes de una aplicación (Activities, Services, Broadcast Receivers, Content Providers) para asegurar una buena experiencia de usuario y la eficiencia del sistema. El sistema puede iniciar, pausar, detener y destruir estos componentes en función de las interacciones del usuario y el estado del sistema.

b) Características:

- **Estados de los Componentes:** Cada tipo de componente tiene un ciclo de vida bien definido con estados específicos (ej., onCreate, onStart, onResume, onPause, onStop, onDestroy para una Activity).
- **Callbacks del Ciclo de Vida:** El framework de Android proporciona métodos de callback que los desarrolladores pueden sobrescribir en sus componentes para realizar acciones específicas en cada etapa del ciclo de vida (ej., guardar el estado en onPause, liberar recursos en onDestroy).
- **Priorización de Procesos:** Android asigna diferentes niveles de prioridad a los procesos de las aplicaciones en función de los componentes que están activos y lo que están haciendo. Los procesos con componentes visibles para el usuario tienen mayor prioridad y es menos probable que sean terminados.
- **Influencia del Usuario y del Sistema:** Las acciones del usuario (abrir una nueva aplicación, volver a una aplicación anterior) y las decisiones del sistema (necesidad de liberar memoria) influyen en las transiciones del ciclo de vida de los componentes.

c) Ejemplos: Cuando el usuario cambia de una aplicación a otra, la Activity de la primera aplicación pasa por los estados onPause() y onStop(). Si el sistema necesita memoria, puede destruir la Activity llamando a onDestroy(). Cuando el usuario vuelve a la aplicación, la Activity pasa por onStart() y onResume(). Los Services pueden ejecutarse en segundo plano independientemente de las Activities y tienen su propio ciclo de vida (onCreate, onStartCommand, onDestroy).

5) Permisos y Seguridad (Permissions and Security)

a) Concepto: El sistema de permisos de Android es un componente clave del entorno de aplicación que controla el acceso de las aplicaciones a recursos sensibles del dispositivo y a ciertas funcionalidades. Esto ayuda a proteger la privacidad del usuario y la integridad del sistema.

b) Características:

- **Permisos Declarados en el Manifiesto:** Las aplicaciones deben declarar en su archivo AndroidManifest.xml los permisos que necesitan para acceder a recursos protegidos (ej., acceso a la cámara, a la ubicación, a los contactos, a internet).
 - **Permisos en Tiempo de Instalación (Normal/Signature):** Algunos permisos (normales y de firma) se otorgan automáticamente en el momento de la instalación.
 - **Permisos en Tiempo de Ejecución (Peligrosos):** Los permisos que podrían afectar la privacidad o la seguridad del usuario (ej., ubicación, cámara, micrófono, contactos) deben ser solicitados y otorgados por el usuario en tiempo de ejecución (a partir de Android 6.0 Marshmallow).
 - **UID y Aislamiento de Procesos:** Como se mencionó anteriormente, el aislamiento de procesos basado en UID es una capa fundamental de seguridad.
 - **SELinux:** Android utiliza Security-Enhanced Linux (SELinux) para aplicar políticas de control de acceso obligatorio (MAC) a nivel del kernel, lo que proporciona una capa adicional de seguridad y aislamiento.
- c) Ejemplos:** Una aplicación de cámara debe solicitar permiso para acceder a la cámara del dispositivo en tiempo de ejecución. Una aplicación de mensajería necesita permiso para leer los contactos del

usuario. El sistema operativo Android utiliza el UID de cada aplicación para controlar su acceso a los archivos y recursos del sistema.